

LAPORAN TUGAS SISTEM OPERASI (PRAKTIKUM)
TUGAS P3.1: PERINTAH DASAR LINUX
CHAPTER 6: SEARCHING FILES AND FILENAMES

Laporan Ini Disusun Untuk Memenuhi Tugas Kuliah
Mata Kuliah : Sistem Operasi



Disusun Oleh :

Nama : Ghaisan Khoirul Badruzaman

NIM : 251524048

Kelas : 2B-D4

PROGRAM STUDI D4 TEKNIK INFORMATIKA
JURUSAN TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
2026

DAFTAR ISI

BAB I PENDAHULUAN	4
1.1 Lingkungan Pengerjaan	4
1.2 Cara Pengerjaan	4
1.3 Perbedaan Urutan Hasil dengan Buku	5
BAB II SOAL GREP	6
2.1 Soal 1: Mencari an di tiga berkas	6
2.2 Soal 2: Kata utuh he	6
2.3 Soal 3: Baris utuh car tanpa peduli besar kecil huruf	7
2.4 Soal 4: Semua baris kecuali yang memuat tea	7
2.5 Soal 5: Memuat do tapi tidak memuat it	8
2.6 Soal 6: Baris cocok beserta satu baris sesudahnya	8
2.7 Soal 7: Konteks sebelum tanpa pemisah grup	9
2.8 Soal 8: Pasangan kurung tanpa kurung di dalamnya	9
2.9 Soal 9: Diawali den atau diakhiri ly	10
2.10 Soal 10: Kata diawali s yang memuat e dan t	10
2.11 Soal 11: Huruf pertama sama dengan huruf terakhir	11
2.12 Soal 12: Mencari *[5] secara literal	11
2.13 Soal 13: hand diikuti s, y, le, atau tidak ada	12
2.14 Soal 14: Field kedua sampai kedua terakhir	12
2.15 Soal 15: Pencarian rekursif ello	13
2.16 Soal 16: Rekursif tanpa folder backups	13
2.17 Soal 17: Memuat blue tapi tidak memuat teal	14
BAB III SOAL FIND DAN LOCATE	15
3.1 Soal 18: Berkas biasa di folder backups	15
3.2 Soal 19: Ekstensi diawali p, s, atau v	15
3.3 Soal 20: Nama tanpa huruf g sampai l	16
3.4 Soal 21: Path memuat direktori diawali p atau d	16

3.5 Soal 22: Direktori bernama memuat b atau d	17
3.6 Soal 23: Direktori tersembunyi	17
3.7 Soal 24: Berkas di kedalaman tepat 2	18
3.8 Soal 25: Beda -mtime dan -atime	19
3.9 Soal 26: Berkas kosong	20
3.10 Soal 27: Menyalin berkas lebih dari 1 byte kecuali .log	20
3.11 Soal 28: Berkas tersembunyi di luar filtered_files	21
3.12 Soal 29: Menghapus filtered_files dan mencari berkas executable	21
3.13 Soal 30: Kapan memakai xargs daripada -exec	22
3.14 Soal 31: Kenapa locate lebih cepat dari find	23
BAB IV PENUTUP	24
4.1 Catatan Hasil	24
4.2 Kesimpulan	24

BAB I PENDAHULUAN

Tugas P3.1 memakai bahan chapter 6 buku *Computing from the Command Line* karya Sundeep Agarwal, yaitu Searching Files and Filenames. Chapter ini membahas tiga perintah: `grep` untuk mencari teks di dalam berkas, `find` untuk mencari berkas berdasarkan nama, ukuran, waktu, dan jenisnya, serta `locate` yang mencari nama berkas lewat database. Laporan ini memuat jawaban 31 soal exercise chapter 6 beserta tangkapan layar hasil percobaannya.

1.1 Lingkungan Pengerjaan

Sama seperti Tugas P1 dan P2, semua percobaan dijalankan di Linux, bukan di macOS, karena buku ini memakai GNU `grep` dan GNU `findutils`. Mesin yang dipakai adalah Ubuntu 24.04 yang berjalan di OrbStack pada MacBook.

Tabel 1.1: Versi perangkat lunak yang dipakai

Komponen	Versi
Sistem operasi	Ubuntu 24.04.5 LTS
Kernel	Linux 6.19.13-orbstack aarch64
Shell	GNU bash 5.2.21
<code>grep</code>	GNU <code>grep</code> 3.11
<code>find</code> dan <code>xargs</code>	GNU <code>findutils</code> 4.9.0
<code>locate</code> dan <code>updatedb</code>	<code>plocate</code> 1.1.19
Locale	C.UTF-8

Image Ubuntu bawaan OrbStack belum punya perintah `locate`, jadi paket `plocate` dipasang dulu untuk soal 31. Sebelum database pertamanya dibuat, `/etc/updatedb.conf` ditambah `virtiofs` pada `PRUNEFS` serta `/Users` dan `/mnt/machines` pada `PRUNEPATHS`. Tujuannya supaya `updatedb` tidak ikut mengindeks seluruh disk macOS yang di-mount OrbStack ke dalam mesin Linux.

1.2 Cara Pengerjaan

1. Soal 1 sampai 14 dikerjakan di `~/cli-computing/example_files/text_files/` karena memakai berkas contoh dari folder itu.
2. Soal 15 sampai 17 dikerjakan di folder `scripts` dengan menjalankan `source grep.sh`, dan soal 18 sampai 30 dengan `source find.sh`, sesuai petunjuk buku. Skrip itu membuat folder `grep_examples` atau `find_examples` beserta isinya. Kalau foldernya sudah ada, skrip hanya masuk ke folder itu, jadi soal 27, 28, dan 29 yang saling bergantung tetap memakai folder yang sama.
3. Folder contoh dibuat baru di soal 15 dan soal 18 supaya isinya sama dengan kondisi awal di buku.
4. Tangkapan layar diambil dari jendela Terminal yang benar-benar menjalankan perintahnya. Judul jendela memperlihatkan `orb -m ubuntu` dan prompt memperlihatkan `ghaisan@ubuntu` beserta folder kerjanya.

1.3 Perbedaan Urutan Hasil dengan Buku

Isi keluaran semua soal cocok dengan buku, tetapi urutan barisnya berbeda pada soal 16, 17, 18, 19, 21, 22, 24, 26, 28, dan 29. `find` dan `grep -r` mencetak hasil sesuai urutan entri direktori yang diberikan filesystem, bukan urutan abjad. Mesin ini memakai `btrfs`, sedangkan buku kemungkinan besar memakai `ext4` yang urutan entrinya berbeda. Kalau hasilnya perluurut, keluarannya bisa diteruskan ke `sort`. Soal 27 juga berbeda urutan, tapi penyebabnya `locale`, dijelaskan di soal itu.

BAB II SOAL GREP

Soal 1 sampai 17 membahas `grep`, mulai dari opsi dasar, regex, sampai pencarian rekursif. `grep` di mesin ini memberi warna merah pada bagian yang cocok, jadi di tangkapan layar terlihat bagian mana yang sebenarnya dicocokkan oleh pola.

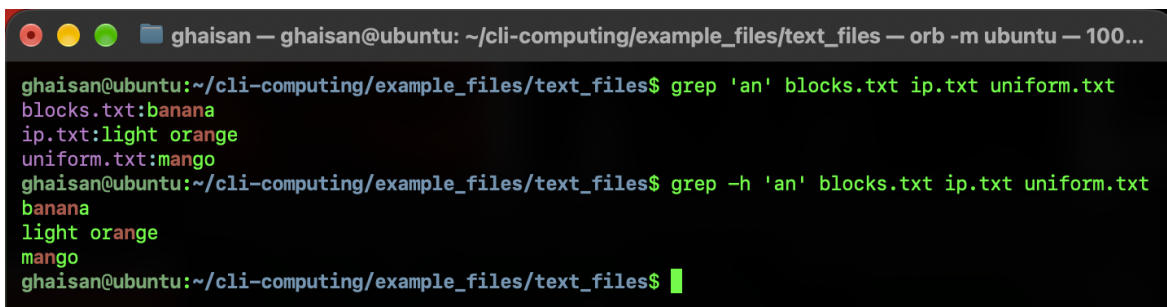
2.1 Soal 1: Mencari an di tiga berkas

Soal. Tampilkan baris yang memuat `an` dari `blocks.txt`, `ip.txt`, dan `uniform.txt`, dengan dan tanpa awalan nama berkas.

Jawaban.

```
grep 'an' blocks.txt ip.txt uniform.txt
grep -h 'an' blocks.txt ip.txt uniform.txt
```

Kalau diberi lebih dari satu berkas, `grep` otomatis menambahkan nama berkas di depan tiap baris. Opsi `-h` menghilangkan awalan itu.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep 'an' blocks.txt ip.txt uniform.txt
blocks.txt:banana
ip.txt:light orange
uniform.txt:mango
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep -h 'an' blocks.txt ip.txt uniform.txt
banana
light orange
mango
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.1: hasil percobaan soal 1

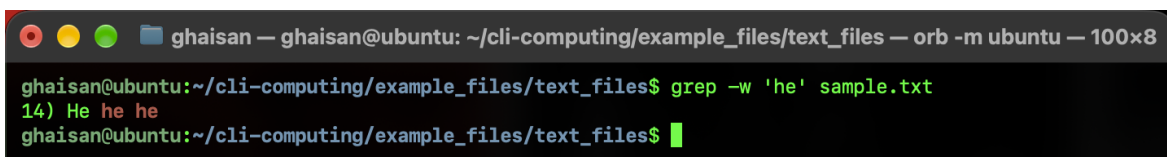
2.2 Soal 2: Kata utuh he

Soal. Tampilkan baris dari `sample.txt` yang memuat kata utuh `he`.

Jawaban.

```
grep -w 'he' sample.txt
```

`-w` membuat `he` hanya cocok kalau berdiri sendiri sebagai kata. Baris seperti `Hi there` tidak ikut walaupun ada huruf `he` di dalam kata `there`.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep -w 'he' sample.txt
14) He he he
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.2: hasil percobaan soal 2

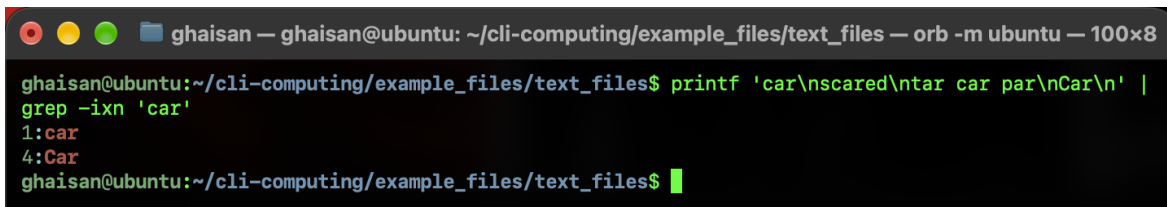
2.3 Soal 3: Baris utuh car tanpa peduli besar kecil huruf

Soal. Cocokkan baris yang isinya persis `car` tanpa membedakan huruf besar dan kecil, beserta nomor barisnya.

Jawaban.

```
printf 'car\nscared\ntar car par\nCar\n' | grep -ixn 'car'
```

`-i` mengabaikan besar kecil huruf, `-x` mengharuskan seluruh baris sama dengan pola, dan `-n` menambahkan nomor baris. Baris `scared` dan `tar car par` sebenarnya memuat `car`, tetapi bukan sebagai baris utuh.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'car\nscared\ntar car par\nCar\n' |  
grep -ixn 'car'  
1:car  
4:Car  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.3: hasil percobaan soal 3

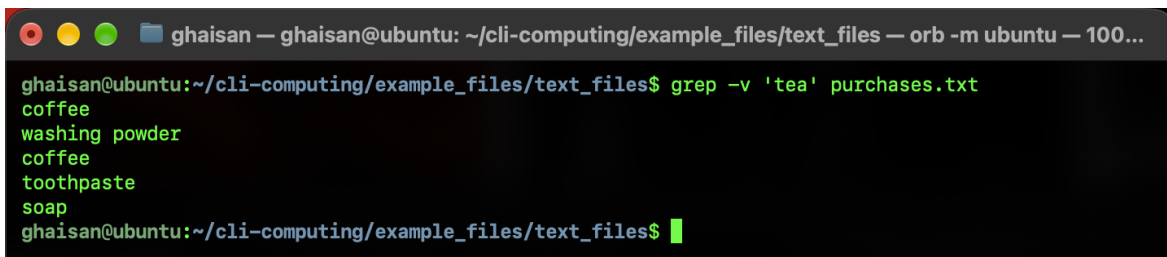
2.4 Soal 4: Semua baris kecuali yang memuat tea

Soal. Tampilkan semua baris `purchases.txt` kecuali yang memuat `tea`.

Jawaban.

```
grep -v 'tea' purchases.txt
```

`-v` membalik pencocokan, jadi yang dicetak justru baris yang tidak memuat pola.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep -v 'tea' purchases.txt  
coffee  
washing powder  
coffee  
toothpaste  
soap  
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.4: hasil percobaan soal 4

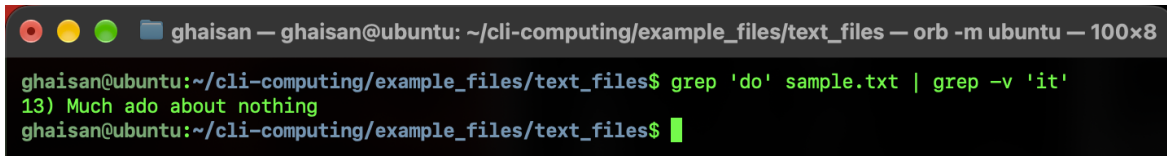
2.5 Soal 5: Memuat do tapi tidak memuat it

Soal. Tampilkan baris `sample.txt` yang memuat `do` tetapi tidak memuat `it`.

Jawaban.

```
grep 'do' sample.txt | grep -v 'it'
```

Dua `grep` disambung pipe. Yang pertama mengambil baris berisi `do`, yang kedua membuang baris berisi `it`. Baris `Just do-it` lolos di tahap pertama tapi gugur di tahap kedua.



```
ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/text_files — orb -m ubuntu — 100x8
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep 'do' sample.txt | grep -v 'it'
13) Much ado about nothing
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.5: hasil percobaan soal 5

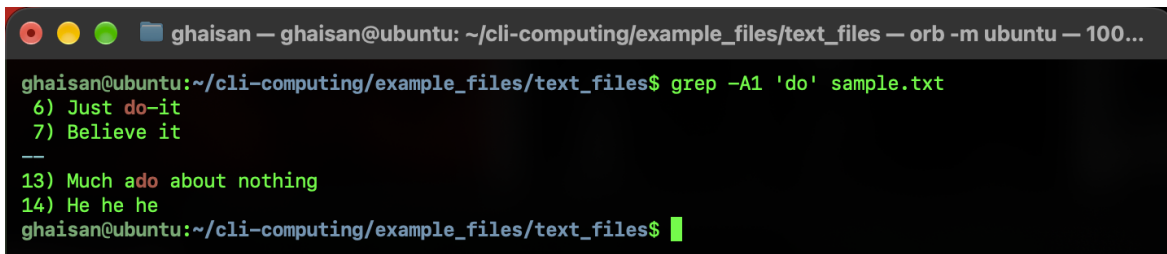
2.6 Soal 6: Baris cocok beserta satu baris sesudahnya

Soal. Tampilkan baris `sample.txt` yang memuat `do` beserta satu baris sesudahnya.

Jawaban.

```
grep -A1 'do' sample.txt
```

`-A1` (after) ikut mencetak satu baris setelah tiap baris yang cocok. Karena dua kelompok hasilnya tidak bersambung, `grep` memisahkannya dengan garis `--`.



```
ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/text_files — orb -m ubuntu — 100...
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep -A1 'do' sample.txt
6) Just do-it
7) Believe it
--
13) Much ado about nothing
14) He he he
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.6: hasil percobaan soal 6

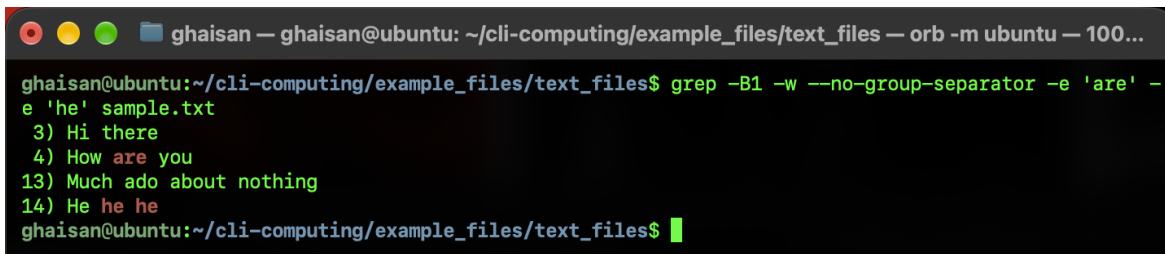
2.7 Soal 7: Konteks sebelum tanpa pemisah grup

Soal. Tampilkan baris yang memuat kata utuh `are` atau `he` beserta satu baris sebelumnya, tanpa pemisah antar grup.

Jawaban.

```
grep -B1 -w --no-group-separator -e 'are' -e 'he' sample.txt
```

`-B1` (before) mencetak satu baris sebelum baris yang cocok. Dua pola ditulis dengan dua `-e` dan `-w` berlaku untuk keduanya. Opsi `--no-group-separator` saya temukan di manual `grep`, fungsinya menghilangkan garis `--` yang muncul di soal 6.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ grep -B1 -w --no-group-separator -e 'are' -e 'he' sample.txt
3) Hi there
4) How are you
13) Much ado about nothing
14) He he he
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.7: hasil percobaan soal 7

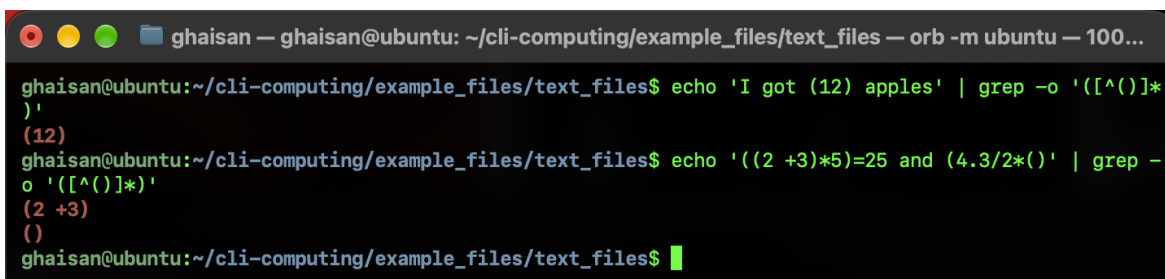
2.8 Soal 8: Pasangan kurung tanpa kurung di dalamnya

Soal. Ambil semua pasangan `()` beserta isinya, asalkan di dalamnya tidak ada karakter kurung lagi.

Jawaban.

```
echo 'I got (12) apples' | grep -o '([^(]*)'
echo '((2 +3)*5)=25 and (4.3/2*())' | grep -o '([^(]*)'
```

`-o` hanya mencetak bagian yang cocok, bukan seluruh baris. `grep` tanpa `-E` memakai BRE, dan di BRE tanda kurung biasa dianggap karakter literal, jadi tidak perlu di-escape. `[^()]` berarti nol atau lebih karakter selain kurung, makanya `()` yang kosong tetap ikut sedangkan `((2 +3)*5)` hanya diambil bagian dalamnya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo 'I got (12) apples' | grep -o '([^(]*)'
(12)
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo '((2 +3)*5)=25 and (4.3/2*())' | grep -o '([^(]*)'
(2 +3)
()
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.8: hasil percobaan soal 8

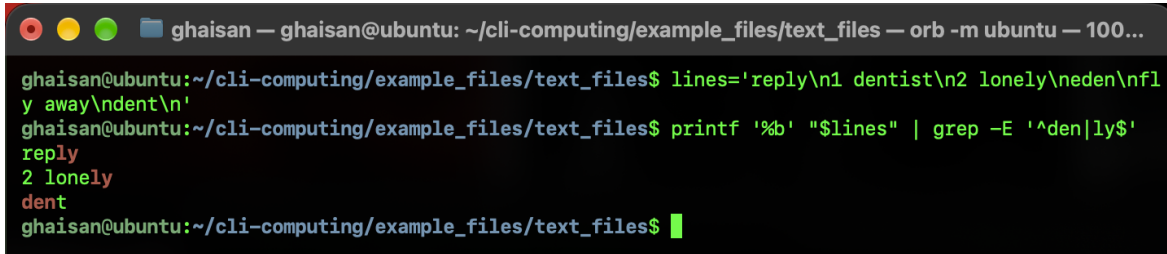
2.9 Soal 9: Diawali den atau diakhiri ly

Soal. Cocokkan baris yang diawali den atau diakhiri ly.

Jawaban.

```
lines='reply\n1 dentist\n2 lonely\neden\nfly away\nndent\n'\nprintf '%b' "$lines" | grep -E '^den|ly$'
```

-E dipakai supaya | berarti pilihan. ^ menandai awal baris dan \$ akhir baris. Baris 1 dentist tidak cocok karena diawali angka, eden tidak diawali den, dan fly away tidak diakhiri ly.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ lines='reply\n1 dentist\n2 lonely\neden\nfly away\nndent\n'\nghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf '%b' "$lines" | grep -E '^den|ly$'\nreply\n2 lonely\ndent\nghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.9: hasil percobaan soal 9

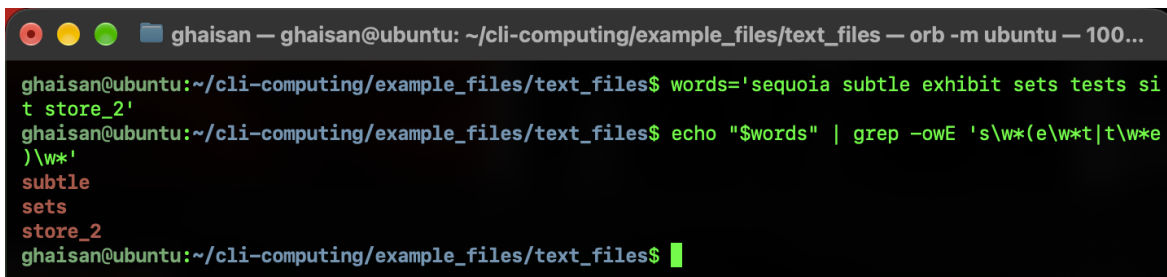
2.10 Soal 10: Kata diawali s yang memuat e dan t

Soal. Ambil kata yang diawali s dan memuat huruf e serta t dengan urutan bebas.

Jawaban.

```
words='sequoia subtle exhibit sets tests sit store_2'\necho "$words" | grep -owE 's\\w*(e\\w*t|t\\w*e)\\w*'
```

Karena urutannya bebas, polanya dibuat dua cabang: e\\w*t untuk e dulu baru t, dan t\\w*e untuk kebalikannya. \\w mencakup huruf, angka, dan garis bawah, jadi store_2 terambil utuh. Kata tests tidak ikut karena diawali t, dan -w memastikan yang diambil selalu kata utuh.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ words='sequoia subtle exhibit sets tests sit store_2'\nghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo "$words" | grep -owE 's\\w*(e\\w*t|t\\w*e)\\w*'\nsubtle\nsets\nstore_2\nghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.10: hasil percobaan soal 10

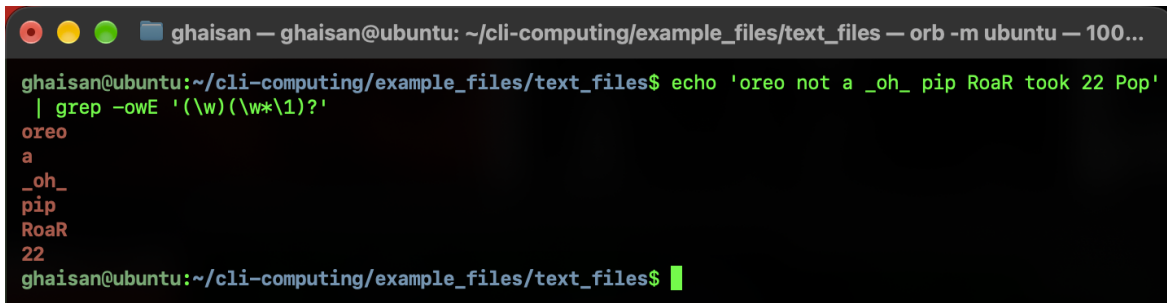
2.11 Soal 11: Huruf pertama sama dengan huruf terakhir

Soal. Ambil kata utuh yang karakter pertama dan terakhirnya sama.

Jawaban.

```
echo 'oreo not a _oh_ pip RoaR took 22 Pop' | grep -owE '(\w)(\w*\1)?'
```

Grup pertama `(\w)` menangkap karakter pertama, lalu `\1` (backreference) memaksa karakter terakhir sama dengan tangkapan itu. Bagian `(\w*\1)?` dibuat opsional supaya kata satu karakter seperti `a` ikut terhitung. `Pop` tidak cocok karena `P` besar dan `p` kecil dianggap berbeda.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo 'oreo not a _oh_ pip RoaR took 22 Pop' | grep -owE '(\w)(\w*\1)?'
```

The terminal output shows the following words: oreo, a, _oh_, pip, RoaR, 22. The word Pop is not listed because its first and last characters (P and p) are different.

Gambar 2.11: hasil percobaan soal 11

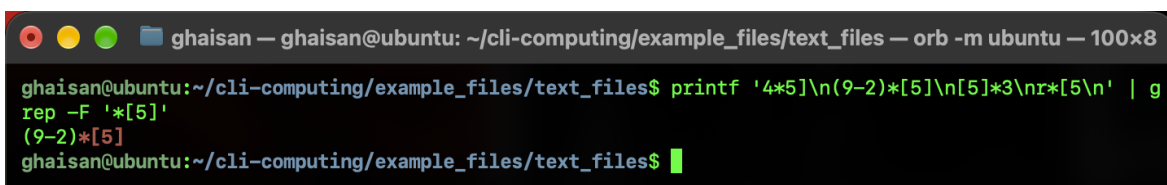
2.12 Soal 12: Mencari `*[5]` secara literal

Soal. Cocokkan baris yang memuat `*[5]` apa adanya.

Jawaban.

```
printf '4*5\n(9-2)*[5]\n[5]*3\nr*[5\n' | grep -F '*[5]'
```

`-F` membuat pola dibaca sebagai string biasa. Tanpa `-F`, `[5]` akan diartikan kelas karakter yang cocok dengan angka 5 saja, dan hasilnya jadi lain.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf '4*5\n(9-2)*[5]\n[5]*3\nr*[5\n' | grep -F '*[5]'
```

The terminal output shows the following lines: (9-2)*[5], [5]*3, and r*[5. The line 4*5 is not listed because the asterisk is not escaped.

Gambar 2.12: hasil percobaan soal 12

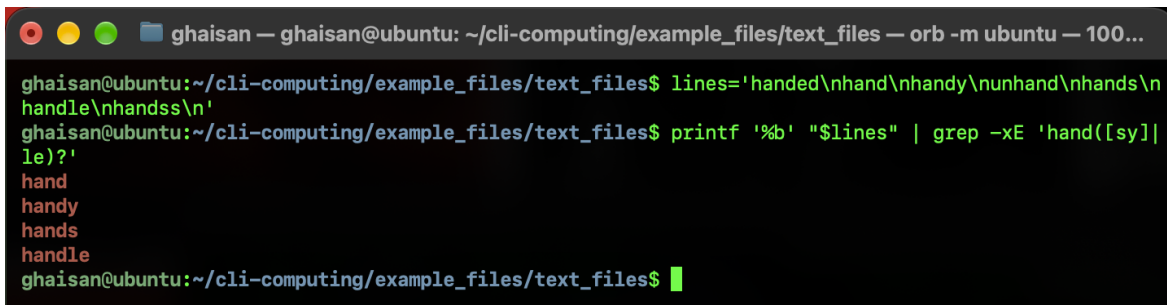
2.13 Soal 13: hand diikuti s, y, le, atau tidak ada

Soal. Cocokkan baris utuh berupa hand yang diikuti s, y, le, atau tidak diikuti apa-apa.

Jawaban.

```
lines='handed\nhand\nhandy\nnunhand\nhands\nhandle\nhandss\n'
printf '%b' "$lines" | grep -xE 'hand([sy]|le)?'
```

-x mengharuskan seluruh baris cocok. ([sy]|le)? berarti satu huruf s atau y, atau le, atau kosong. handss gagal karena ada dua s, sedangkan handed dan unhand gagal di awal atau akhirnya.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ lines='handed\nhand\nhandy\nnunhand\nhands\nhandle\nhandss\n'
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf '%b' "$lines" | grep -xE 'hand([sy]|le)?'
hand
handy
hands
handle
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.13: hasil percobaan soal 13

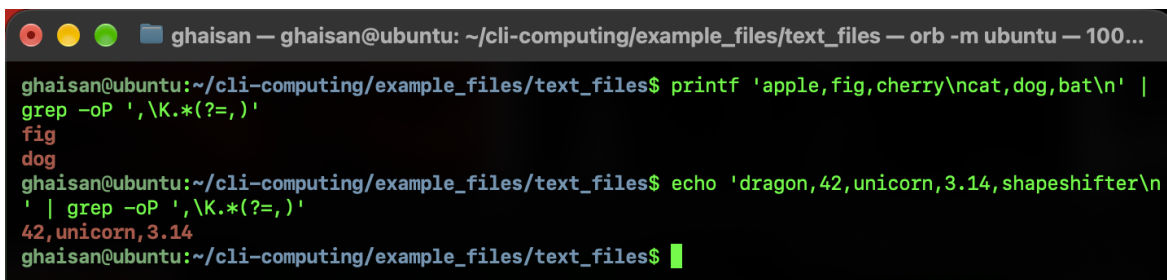
2.14 Soal 14: Field kedua sampai kedua terakhir

Soal. Ambil field kedua sampai field kedua dari belakang pada baris yang dipisah koma.

Jawaban.

```
printf 'apple,fig,cherry\ncat,dog,bat\n' | grep -oP ',\K.*(?:=,)'
echo 'dragon,42,unicorn,3.14,shapeshifter\n' | grep -oP ',\K.*(?:=,)'
```

-P mengaktifkan regex gaya Perl. \K membuang bagian yang sudah cocok sebelumnya (koma pertama) dari hasil, sedangkan (?:=,) adalah lookahead: harus ada koma sesudahnya tetapi koma itu tidak ikut tercetak. Karena .* rakus, pencocokan berhenti di koma terakhir. Pada contoh kedua, echo tanpa -e mencetak \n apa adanya sehingga ikut jadi bagian field terakhir, tapi field terakhir memang dibuang jadi hasilnya tidak terpengaruh.



```
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ printf 'apple,fig,cherry\ncat,dog,bat\n' |
grep -oP ',\K.*(?:=,)'
fig
dog
ghaisan@ubuntu:~/cli-computing/example_files/text_files$ echo 'dragon,42,unicorn,3.14,shapeshifter\n'
| grep -oP ',\K.*(?:=,)'
42,unicorn,3.14
ghaisan@ubuntu:~/cli-computing/example_files/text_files$
```

Gambar 2.14: hasil percobaan soal 14

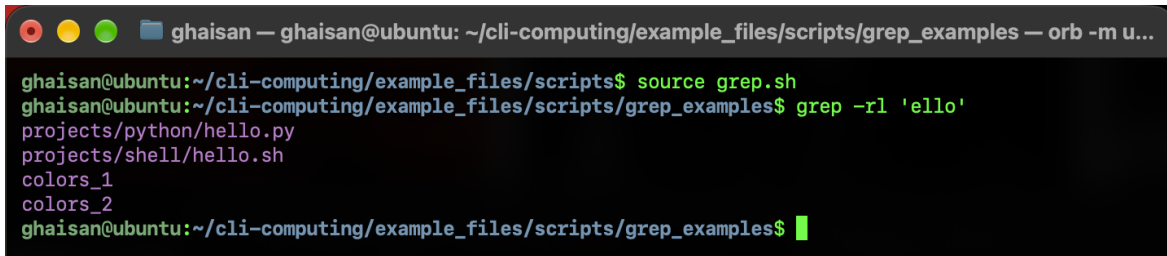
2.15 Soal 15: Pencarian rekursif ello

Soal. Cari berkas yang memuat `ello` secara rekursif.

Jawaban.

```
source grep.sh
grep -rl 'ello'
```

`-r` menelusuri folder sampai ke sub-direktornya dan `-l` hanya mencetak nama berkas yang cocok. `colors_1` dan `colors_2` ikut karena memuat kata `yellow`.

A terminal window with a dark background and light green text. The window title is "ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/grep_examples — orb -m u...". The prompt is "ghaisan@ubuntu:~/cli-computing/example_files/scripts\$". The user enters "source grep.sh". The prompt changes to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples\$". The user enters "grep -rl 'ello'". The output is "projects/python/hello.py", "projects/shell/hello.sh", "colors_1", and "colors_2". The prompt returns to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples\$".

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source grep.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples$ grep -rl 'ello'
projects/python/hello.py
projects/shell/hello.sh
colors_1
colors_2
ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples$
```

Gambar 2.15: hasil percobaan soal 15

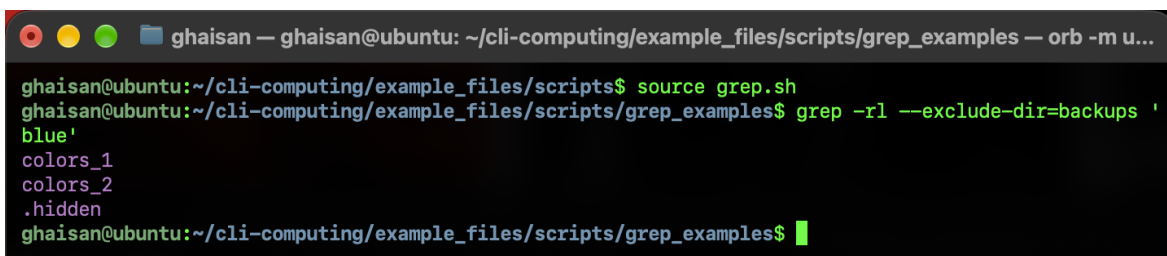
2.16 Soal 16: Rekursif tanpa folder backups

Soal. Cari berkas yang memuat `blue` secara rekursif, tetapi jangan mencari di folder `backups`.

Jawaban.

```
source grep.sh
grep -rl --exclude-dir=backups 'blue'
```

`--exclude-dir=backups` melewati folder `backups`, jadi `backups/color list.txt` yang juga memuat `blue` tidak ikut. Urutan hasilnya berbeda dengan buku, penjelasannya ada di bagian 1.3.

A terminal window with a dark background and light green text. The window title is "ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/grep_examples — orb -m u...". The prompt is "ghaisan@ubuntu:~/cli-computing/example_files/scripts\$". The user enters "source grep.sh". The prompt changes to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples\$". The user enters "grep -rl --exclude-dir=backups 'blue'". The output is "colors_1", "colors_2", and ".hidden". The prompt returns to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples\$".

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source grep.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples$ grep -rl --exclude-dir=backups 'blue'
colors_1
colors_2
.hidden
ghaisan@ubuntu:~/cli-computing/example_files/scripts/grep_examples$
```

Gambar 2.16: hasil percobaan soal 16

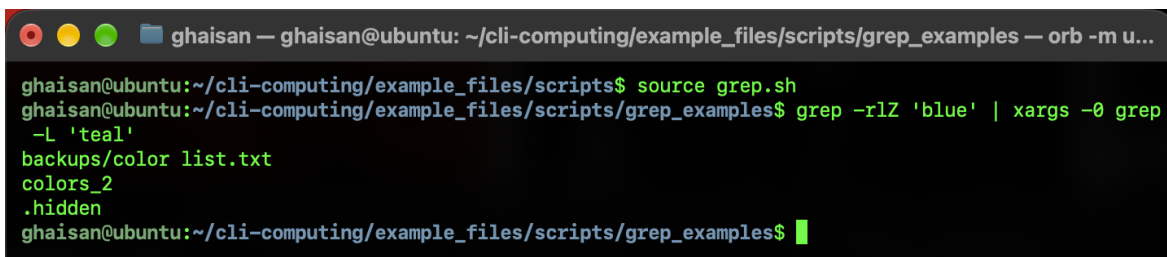
2.17 Soal 17: Memuat blue tapi tidak memuat teal

Soal. Cari berkas yang memuat blue secara rekursif, kecuali yang juga memuat teal.

Jawaban.

```
source grep.sh
grep -rLZ 'blue' | xargs -0 grep -L 'teal'
```

`-L` kebalikan `-l`, yaitu mencetak berkas yang tidak memuat pola. Daftar berkas berisi blue diteruskan ke `grep` kedua lewat `xargs`. Karena ada nama berkas yang mengandung spasi (`color list.txt`), nama berkas dipisah null byte dengan `-Z` dan dibaca `xargs -0`. Kalau tidak, `xargs` akan memecahnya jadi dua nama berkas yang tidak ada. `colors_1` gugur karena memuat teal.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source grep.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ grep -rLZ 'blue' | xargs -0 grep -L 'teal'
backups/color list.txt
colors_2
.hidden
ghaisan@ubuntu:~/cli-computing/example_files/scripts$
```

Gambar 2.17: hasil percobaan soal 17

BAB III SOAL FIND DAN LOCATE

Soal 18 sampai 30 membahas `find` dan dikerjakan di folder `find_examples`, sedangkan soal 31 membahas `locate` dan dikerjakan di folder `home`.

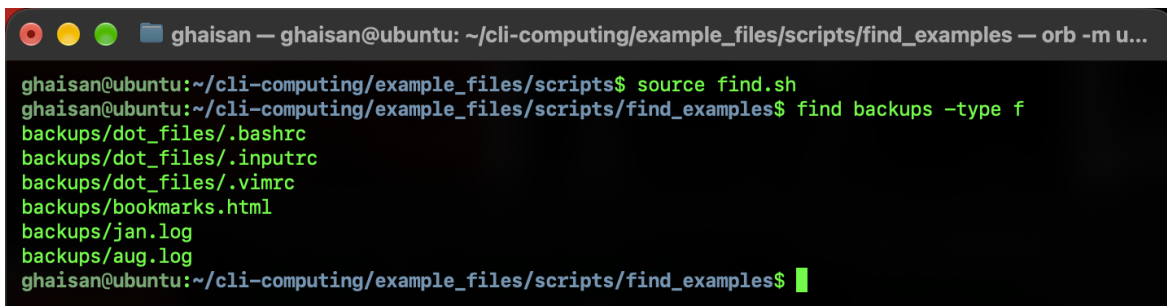
3.1 Soal 18: Berkas biasa di folder backups

Soal. Cari semua berkas biasa di dalam folder `backups`.

Jawaban.

```
source find.sh
find backups -type f
```

Titik awal pencarian diberikan `backups`, dan `-type f` membatasi hasil ke berkas biasa sehingga folder `dot_files` tidak ikut tercetak.

A terminal window titled 'ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...' shows the following commands and output:

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find backups -type f
backups/dot_files/.bashrc
backups/dot_files/.inputrc
backups/dot_files/.vimrc
backups/bookmarks.html
backups/jan.log
backups/aug.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.1: hasil percobaan soal 18

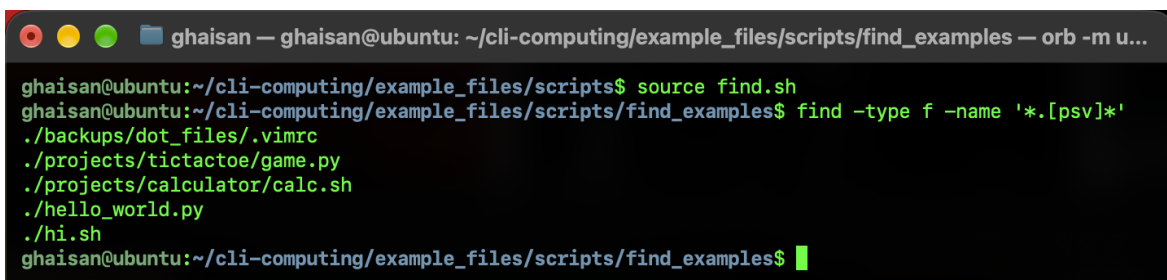
3.2 Soal 19: Ekstensi diawali p, s, atau v

Soal. Cari berkas biasa yang ekstensinya diawali `p`, `s`, atau `v`.

Jawaban.

```
source find.sh
find -type f -name '.*[psv]*'
```

`-name` mencocokkan nama berkas tanpa path dengan wildcard seperti di shell. Polanya dikutip supaya tidak dilebarkan shell lebih dulu. `.vimrc` ikut karena `*` boleh kosong, sehingga titik di depan `.vimrc` terbaca sebagai titik sebelum ekstensi.

A terminal window titled 'ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...' shows the following commands and output:

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -name '.*[psv]*'
./backups/dot_files/.vimrc
./projects/tictactoe/game.py
./projects/calculator/calc.sh
./hello_world.py
./hi.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.2: hasil percobaan soal 19

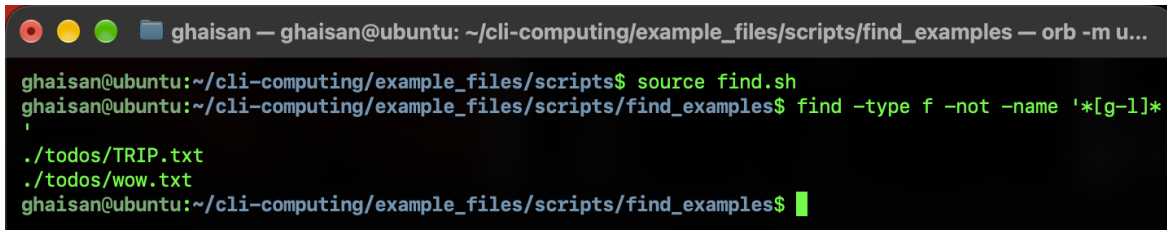
3.3 Soal 20: Nama tanpa huruf g sampai l

Soal. Cari berkas biasa yang namanya tidak memuat huruf kecil g sampai l.

Jawaban.

```
source find.sh
find -type f -not -name '*[g-l]*'
```

[g-l] cocok dengan nama yang memuat minimal satu huruf g sampai l, lalu -not membalikinya. TRIP.txt lolos karena huruf I-nya huruf besar.

A terminal window with a dark background and light green text. The window title is 'ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...'. The user enters 'source find.sh' and then 'find -type f -not -name '*[g-l]*''. The output shows two files: './todos/TRIP.txt' and './todos/wow.txt'. The prompt returns to the user's input line.

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -not -name '*[g-l]*'
./todos/TRIP.txt
./todos/wow.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.3: hasil percobaan soal 20

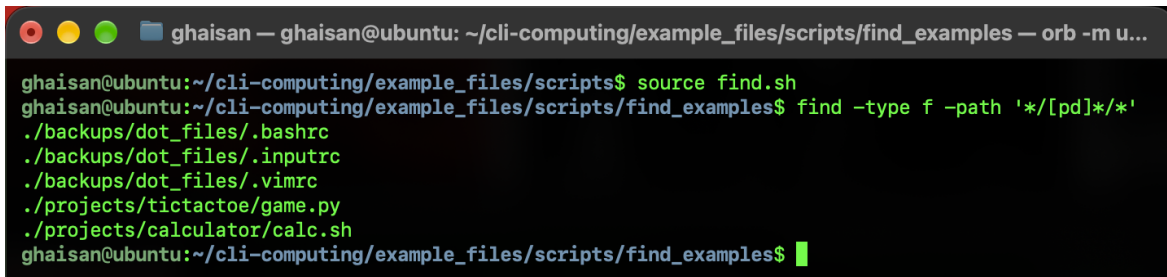
3.4 Soal 21: Path memuat direktori diawali p atau d

Soal. Cari berkas biasa yang path-nya memuat minimal satu nama direktori diawali p atau d.

Jawaban.

```
source find.sh
find -type f -path '*/[pd]*/*'
```

-path mencocokkan seluruh path, bukan cuma nama berkas. Bedanya dengan wildcard shell, * di sini boleh melewati /. Pola */[pd]*/* berarti ada komponen yang diawali p atau d dan masih diikuti /, artinya komponen itu direktori.

A terminal window with a dark background and light green text. The window title is 'ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...'. The user enters 'source find.sh' and then 'find -type f -path '*/[pd]*/*''. The output shows several files: './backups/dot_files/.bashrc', './backups/dot_files/.inputrc', './backups/dot_files/.vimrc', './projects/tictactoe/game.py', and './projects/calculator/calc.sh'. The prompt returns to the user's input line.

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -path '*/[pd]*/*'
./backups/dot_files/.bashrc
./backups/dot_files/.inputrc
./backups/dot_files/.vimrc
./projects/tictactoe/game.py
./projects/calculator/calc.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.4: hasil percobaan soal 21

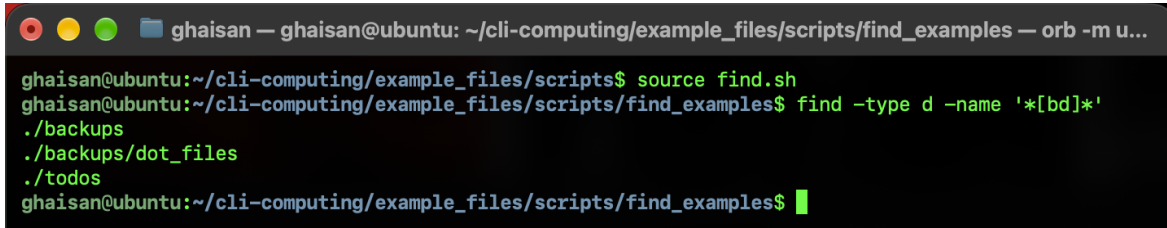
3.5 Soal 22: Direktori bernama memuat b atau d

Soal. Cari semua direktori yang namanya memuat b atau d.

Jawaban.

```
source find.sh
find -type d -name '*[bd]*'
```

`-type d` membatasi ke direktori, dan `*[bd]*` cocok dengan nama yang memuat b atau d di posisi mana pun.

A terminal window with a dark background. The title bar shows a window icon and the text "ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...". The terminal content shows the user "ghaisan" at "ubuntu" in the directory "~/cli-computing/example_files/scripts". They run "source find.sh" and then "find -type d -name '*[bd]*'". The output lists three directories: "./backups", "./backups/dot_files", and "./todos". The prompt returns to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples\$".

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type d -name '*[bd]*'
./backups
./backups/dot_files
./todos
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.5: hasil percobaan soal 22

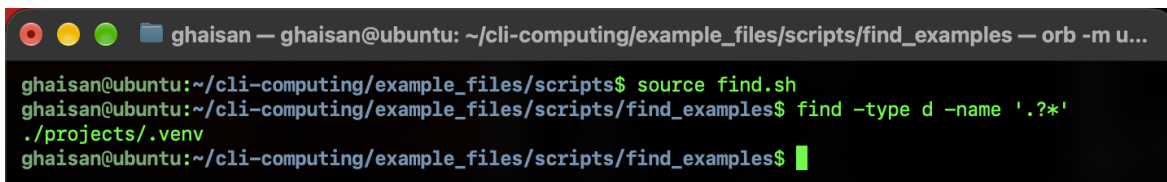
3.6 Soal 23: Direktori tersembunyi

Soal. Cari semua direktori tersembunyi.

Jawaban.

```
source find.sh
find -type d -name '.*'
```

Awalnya saya pakai `-name '.*'`, tapi hasilnya ikut memuat `.` yaitu direktori awal pencarian, karena namanya memang diawali titik. Pola `.?*` mengharuskan minimal satu karakter lagi setelah titik, jadi `.` tidak ikut. Cara lain yang hasilnya sama adalah `find -mindepth 1 -type d -name '.*'`.

A terminal window with a dark background. The title bar shows a window icon and the text "ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...". The terminal content shows the user "ghaisan" at "ubuntu" in the directory "~/cli-computing/example_files/scripts". They run "source find.sh" and then "find -type d -name '.*'". The output lists one directory: "./projects/.venv". The prompt returns to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples\$".

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type d -name '.*'
./projects/.venv
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.6: hasil percobaan soal 23

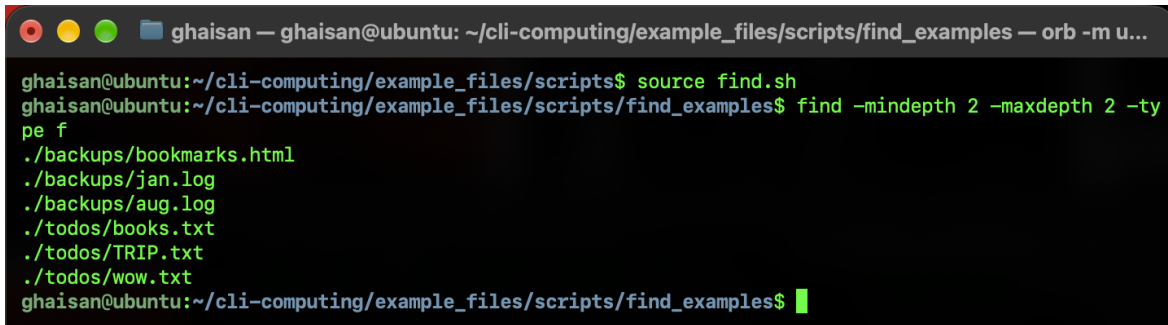
3.7 Soal 24: Berkas di kedalaman tepat 2

Soal. Cari berkas biasa yang berada tepat di kedalaman 2.

Jawaban.

```
source find.sh  
find -mindepth 2 -maxdepth 2 -type f
```

Kedalaman 1 adalah isi langsung folder awal, kedalaman 2 adalah isi sub-folder. Dengan `-mindepth 2` dan `-maxdepth 2` sekaligus, hanya kedalaman 2 yang tersisa.

A terminal window with a dark background and light green text. The window title is "ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...". The prompt is "ghaisan@ubuntu:~/cli-computing/example_files/scripts\$". The first command is "source find.sh". The second command is "ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples\$ find -mindepth 2 -maxdepth 2 -type f". The output lists six files: "./backups/bookmarks.html", "./backups/jan.log", "./backups/aug.log", "./todos/books.txt", "./todos/TRIP.txt", and "./todos/wow.txt". The prompt returns to "ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples\$".

```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh  
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -mindepth 2 -maxdepth 2 -type f  
./backups/bookmarks.html  
./backups/jan.log  
./backups/aug.log  
./todos/books.txt  
./todos/TRIP.txt  
./todos/wow.txt  
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.7: hasil percobaan soal 24

3.8 Soal 25: Beda -mtime dan -atime

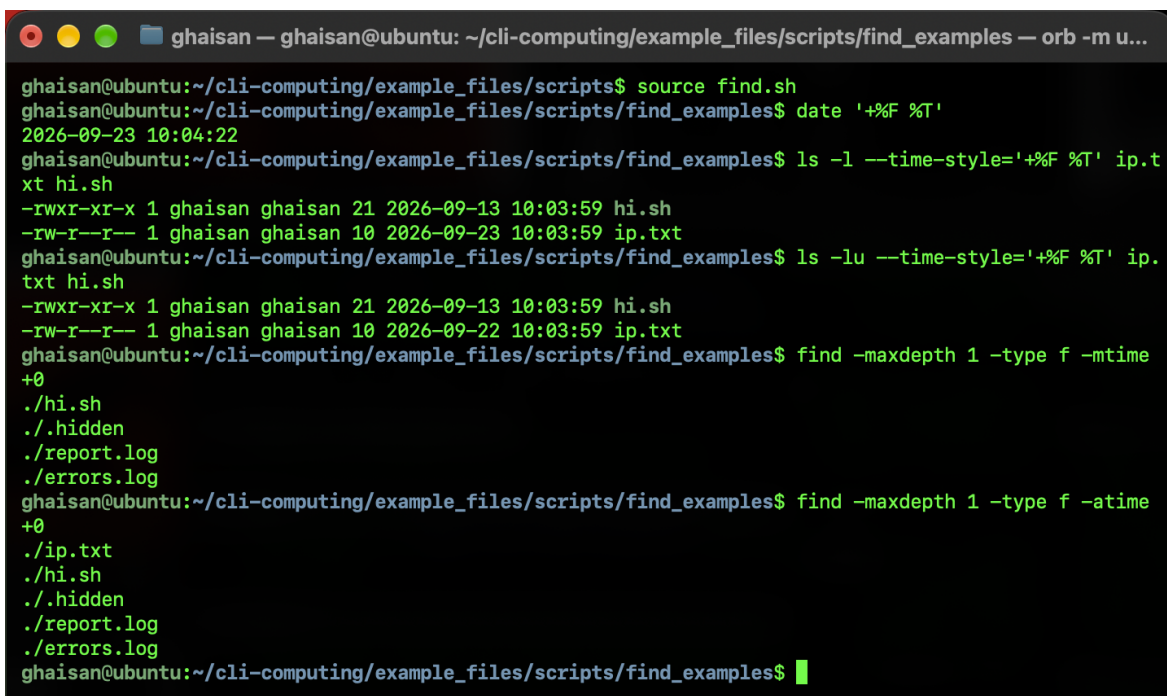
Soal. Apa beda `find -mtime` dan `find -atime`, dan periode waktu apa yang dipakai?

Jawaban. `-mtime` melihat waktu terakhir isi berkas diubah (modification time), sedangkan `-atime` melihat waktu terakhir berkas dibaca (access time). Satuannya hari, yaitu periode 24 jam dihitung mundur dari saat perintah dijalankan, dan sisa pecahannya dibuang. Jadi `-mtime 0` berarti diubah kurang dari 24 jam lalu, `-mtime +0` berarti minimal 24 jam lalu, dan `-mtime -3` berarti kurang dari 3 hari. Kalau butuh satuan menit, pasangannya adalah `-mmin` dan `-amin`.

Percobaannya memakai `ip.txt`. Berkas ini baru dibuat oleh `find.sh` sehingga `mtime`-nya hari ini, tetapi `atime`-nya sengaja dimundurkan satu hari dengan `touch -a. ls -l` menampilkan `mtime` dan `ls -lu` menampilkan `atime`. Hasilnya `ip.txt` hanya muncul di `-atime +0`, sedangkan `hi.sh` yang `mtime` dan `atime`-nya sama-sama 10 hari lalu muncul di keduanya.

Satu hal yang saya temukan waktu mengecek: filesystem root mesin ini di-mount dengan opsi `noatime`, jadi membaca berkas tidak mengubah `atime` sama sekali. Nilai `atime` di percobaan ini murni hasil `touch -a` dari `find.sh`.

```
source find.sh
date '+%F %T'
ls -l --time-style='+%F %T' ip.txt hi.sh
ls -lu --time-style='+%F %T' ip.txt hi.sh
find -maxdepth 1 -type f -mtime +0
find -maxdepth 1 -type f -atime +0
```



```
ghaisan — ghaisan@ubuntu: ~/cli-computing/example_files/scripts/find_examples — orb -m u...

ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ date '+%F %T'
2026-09-23 10:04:22
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ ls -l --time-style='+%F %T' ip.txt hi.sh
-rwxr-xr-x 1 ghaisan ghaisan 21 2026-09-13 10:03:59 hi.sh
-rw-r--r-- 1 ghaisan ghaisan 10 2026-09-23 10:03:59 ip.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ ls -lu --time-style='+%F %T' ip.txt hi.sh
-rwxr-xr-x 1 ghaisan ghaisan 21 2026-09-13 10:03:59 hi.sh
-rw-r--r-- 1 ghaisan ghaisan 10 2026-09-22 10:03:59 ip.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -maxdepth 1 -type f -mtime +0
./hi.sh
./hidden
./report.log
./errors.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -maxdepth 1 -type f -atime +0
./ip.txt
./hi.sh
./hidden
./report.log
./errors.log
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.8: hasil percobaan soal 25

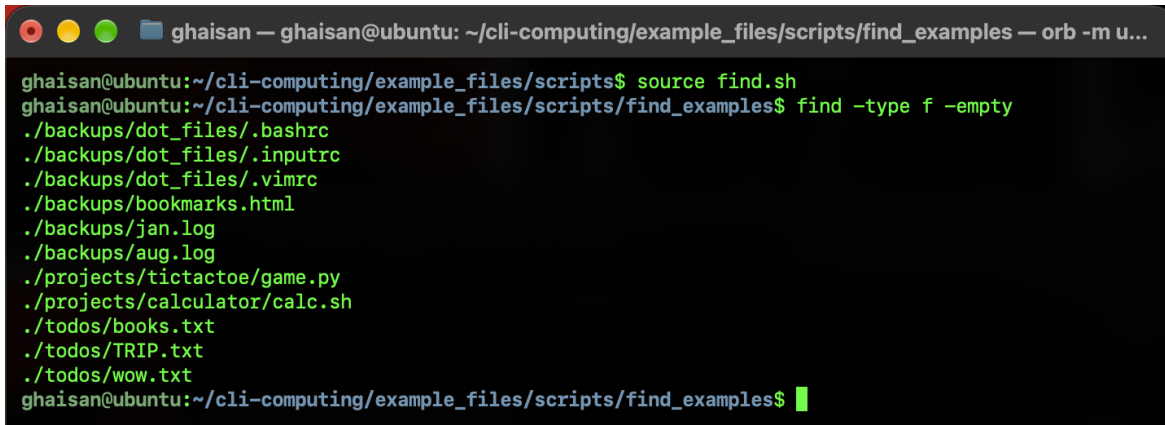
3.9 Soal 26: Berkas kosong

Soal. Cari semua berkas biasa yang kosong.

Jawaban.

```
source find.sh
find -type f -empty
```

-empty cocok dengan berkas berukuran 0 byte dan juga direktori kosong, makanya ditambah -type f supaya direktori kosong seperti .venv tidak ikut.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -empty
./backups/dot_files/.bashrc
./backups/dot_files/.inputrc
./backups/dot_files/.vimrc
./backups/bookmarks.html
./backups/jan.log
./backups/aug.log
./projects/tictactoe/game.py
./projects/calculator/calc.sh
./todos/books.txt
./todos/TRIP.txt
./todos/wow.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.9: hasil percobaan soal 26

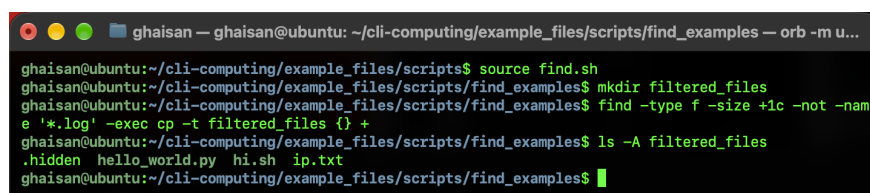
3.10 Soal 27: Menyalin berkas lebih dari 1 byte kecuali .log

Soal. Buat direktori `filtered_files`, lalu salin semua berkas biasa berukuran lebih dari 1 byte yang namanya tidak berakhiran `.log`.

Jawaban.

```
source find.sh
mkdir filtered_files
find -type f -size +1c -not -name '*.log' -exec cp -t filtered_files {} +
ls -A filtered_files
```

-size +1c berarti lebih dari 1 byte. Akhiran c penting, karena tanpa satuan find menghitung dalam blok 512 byte sehingga +1 artinya lebih dari satu blok. -exec cp -t filtered_files {} + mengganti {} dengan daftar nama berkas lalu menjalankan satu kali cp, dan -t dipakai karena folder tujuan harus ditulis sebelum daftar berkasnya. Urutan ls -A berbeda dengan buku karena locale mesin ini C.UTF-8, yang mengurutkan titik lebih dulu dari huruf.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ mkdir filtered_files
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -size +1c -not -name '*.log' -exec cp -t filtered_files {} +
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ ls -A filtered_files
.hidden hello_world.py hi.sh ip.txt
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.10: hasil percobaan soal 27

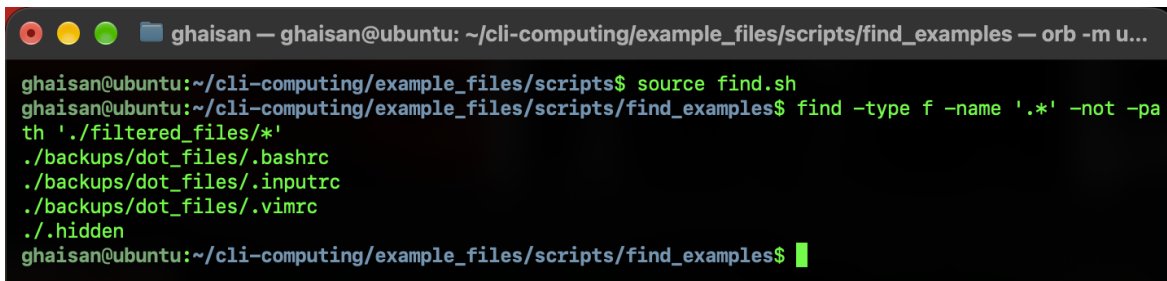
3.11 Soal 28: Berkas tersembunyi di luar filtered_files

Soal. Cari semua berkas tersembunyi, kecuali yang ada di dalam `filtered_files`.

Jawaban.

```
source find.sh
find -type f -name '.*' -not -path './filtered_files/*'
```

`-name '.*'` mengambil nama yang diawali titik, lalu `-not -path './filtered_files/*'` membuang isi folder `filtered_files`. Bisa juga memakai `-prune`, tapi versi ini lebih mudah dibaca.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -name '.*' -not -path './filtered_files/*'
./backups/dot_files/.bashrc
./backups/dot_files/.inputrc
./backups/dot_files/.vimrc
./hidden
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.11: hasil percobaan soal 28

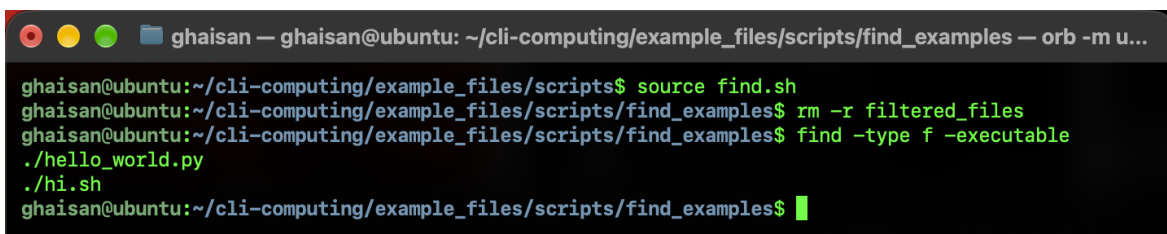
3.12 Soal 29: Menghapus filtered_files dan mencari berkas executable

Soal. Hapus `filtered_files`, lalu cari cara menampilkan berkas yang executable saja.

Jawaban.

```
source find.sh
rm -r filtered_files
find -type f -executable
```

`rm -r` menghapus folder beserta isinya. Dari manual `find`, opsi `-executable` cocok dengan berkas yang boleh dijalankan oleh pengguna yang sedang login. Hasilnya dua skrip yang diberi `chmod +x` oleh `find.sh`.



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ rm -r filtered_files
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ find -type f -executable
./hello_world.py
./hi.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.12: hasil percobaan soal 29

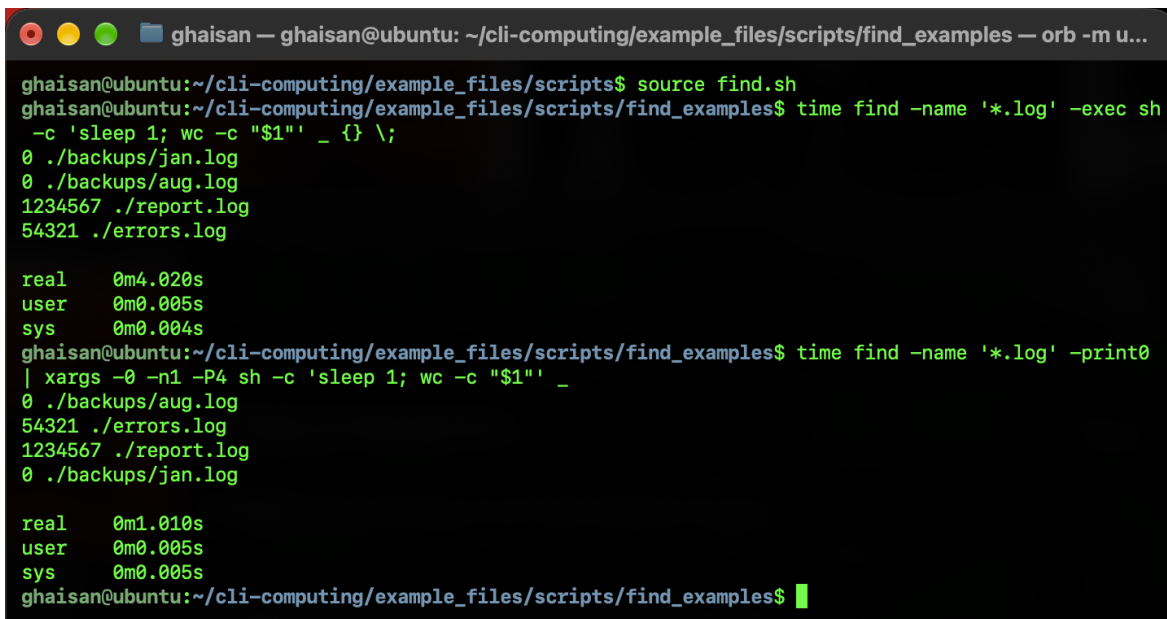
3.13 Soal 30: Kapan memakai xargs daripada -exec

Soal. Sebutkan minimal satu kasus yang lebih cocok memakai pipe `find` ke `xargs` daripada opsi `find -exec`.

Jawaban. Contoh yang paling terasa adalah menjalankan perintah secara paralel. `find -exec` menjalankan perintahnya satu per satu, sedangkan `xargs -P` bisa menjalankan beberapa proses sekaligus. Percobaan di bawah memberi tiap berkas `.log` pekerjaan yang makan waktu 1 detik (`sleep 1`, lalu `wc -c`). Dengan `-exec` totalnya sekitar 4 detik untuk 4 berkas, dengan `xargs -n1 -P4` cukup sekitar 1 detik. Urutan keluaran versi paralel jadi acak karena proses yang selesai duluan yang mencetak duluan.

Kegunaan lain, `xargs` bisa menerima daftar berkas dari perintah apa pun, bukan hanya `find`, seperti pada soal 17 yang memakai `grep -rlZ | xargs -0`. Pasangan `-print0` dan `-0` dipakai supaya nama berkas yang mengandung spasi tetap aman.

```
source find.sh
time find -name '*.log' -exec sh -c 'sleep 1; wc -c "$1" _ {} \;
time find -name '*.log' -print0 | xargs -0 -n1 -P4 sh -c 'sleep 1; wc -c "$1" _
```



```
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples — orb -m u...

ghaisan@ubuntu:~/cli-computing/example_files/scripts$ source find.sh
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ time find -name '*.log' -exec sh
-c 'sleep 1; wc -c "$1" _ {} \;
0 ./backups/jan.log
0 ./backups/aug.log
1234567 ./report.log
54321 ./errors.log

real    0m4.020s
user    0m0.005s
sys     0m0.004s
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$ time find -name '*.log' -print0
| xargs -0 -n1 -P4 sh -c 'sleep 1; wc -c "$1" _
0 ./backups/aug.log
54321 ./errors.log
1234567 ./report.log
0 ./backups/jan.log

real    0m1.010s
user    0m0.005s
sys     0m0.005s
ghaisan@ubuntu:~/cli-computing/example_files/scripts/find_examples$
```

Gambar 3.13: hasil percobaan soal 30

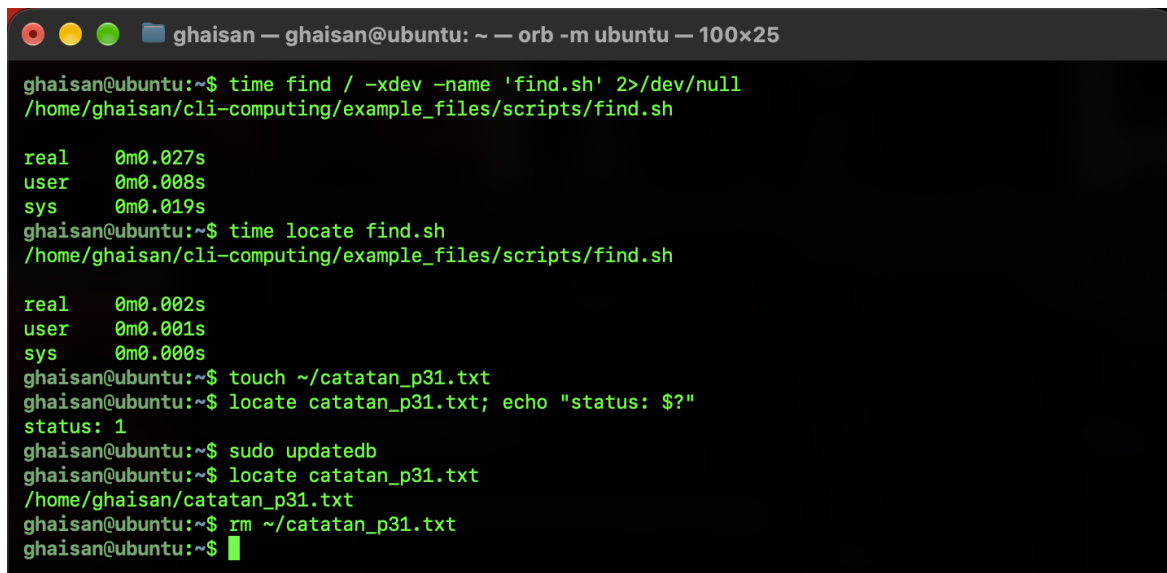
3.14 Soal 31: Kenapa locate lebih cepat dari find

Soal. Bagaimana `locate` bisa bekerja lebih cepat daripada perintah `find` yang setara?

Jawaban. `find` menelusuri pohon direktori saat itu juga, membaca isi tiap folder satu per satu. `locate` tidak menelusuri apa-apa. Ia mencari di database berisi daftar path yang sudah dibuat sebelumnya oleh `updatedb`. Di Ubuntu 24.04, `locate` disediakan paket `plocate` yang databasenya berupa indeks trigram (potongan tiga karakter), sehingga pencarian potongan nama pun tetap cepat.

Kekurangannya, hasil `locate` hanya sebar database terakhir. Pada percobaan di bawah, berkas yang baru dibuat tidak ditemukan (status keluar 1) sampai `updatedb` dijalankan ulang. Biasanya `updatedb` dijalankan otomatis sekali sehari oleh timer `systemd` atau `cron`, sedangkan di mesin percobaan ini dijalankan manual. Selisih waktunya di mesin ini kecil karena filesystem root-nya sendiri kecil, tetapi `locate` tetap lebih dari sepuluh kali lebih cepat, dan selisihnya makin besar kalau jumlah berkasnya makin banyak.

```
time find / -xdev -name 'find.sh' 2>/dev/null
time locate find.sh
touch ~/catatan_p31.txt
locate catatan_p31.txt; echo "status: $?"
sudo updatedb
locate catatan_p31.txt
rm ~/catatan_p31.txt
```



```
ghaisan@ubuntu: ~ — orb -m ubuntu — 100x25

ghaisan@ubuntu:~$ time find / -xdev -name 'find.sh' 2>/dev/null
/home/ghaisan/cli-computing/example_files/scripts/find.sh

real    0m0.027s
user    0m0.008s
sys     0m0.019s
ghaisan@ubuntu:~$ time locate find.sh
/home/ghaisan/cli-computing/example_files/scripts/find.sh

real    0m0.002s
user    0m0.001s
sys     0m0.000s
ghaisan@ubuntu:~$ touch ~/catatan_p31.txt
ghaisan@ubuntu:~$ locate catatan_p31.txt; echo "status: $?"
status: 1
ghaisan@ubuntu:~$ sudo updatedb
ghaisan@ubuntu:~$ locate catatan_p31.txt
/home/ghaisan/catatan_p31.txt
ghaisan@ubuntu:~$ rm ~/catatan_p31.txt
ghaisan@ubuntu:~$
```

Gambar 3.14: hasil percobaan soal 31

BAB IV PENUTUP

4.1 Catatan Hasil

1. Semua 28 soal yang punya contoh keluaran menghasilkan isi yang sama dengan buku. Tiga soal lainnya (25, 30, 31) berupa pertanyaan, dan jawabannya dilengkapi percobaan.
2. Pola wildcard untuk `find -name` dan `-path` harus dikutip. Kalau tidak, shell bisa melebarkannya duluan menjadi nama berkas di folder aktif, dan `find` menerima pola yang salah.
3. Pola `-name '.*'` ikut mencocokkan direktori awal `.`, jadi untuk mencari direktori tersembunyi perlu `.*?` atau `-mindepth 1`.
4. Nama berkas yang mengandung spasi merusak pipe biasa ke `xargs`. Pasangan `grep -Z` atau `find -print0` dengan `xargs -0` menyelesaikannya.
5. Satuan `-size` tanpa huruf adalah blok 512 byte, bukan byte, sehingga untuk byte harus ditulis akhiran `c`.

4.2 Kesimpulan

`grep` dan `find` saling melengkapi. `grep` menjawab berkas mana yang isinya memuat sesuatu, sedangkan `find` menjawab berkas mana yang nama, ukuran, waktu, atau jenisnya memenuhi syarat. Keduanya bisa digabung lewat `xargs` atau `-exec`, seperti pada soal 17 dan 30. `locate` lebih cepat karena hanya membaca database, dengan konsekuensi hasilnya bisa ketinggalan dari kondisi disk yang sebenarnya.